

Annexe 2 : Conception d'un agent RPA **autonome pour l'extraction documentaire** **(Projet Doc-Sat)**

1. Contexte technique et contraintes métier	2
2. Architecture de la solution	2
A. Vérifications de départ et gestion des droits	2
B. Mode autonome et configuration	3
C. Moteur RPA, résilience et fichiers composites	4
D. Maintenance disque et Audit de production	5
3. Preuves matérielles	6
4. Bilan	8

1. Contexte technique et contraintes métier

Le projet "**Doc-Sat dématérialisés**" vise à rendre les documents techniques accessibles aux techniciens sur le terrain, directement depuis leurs tablettes. Cependant, l'interface du logiciel intranet actuel n'est pas adaptée à cet usage mobile. Comme ce logiciel est géré au niveau national, attendre une mise à jour officielle aurait pris beaucoup trop de temps (malgré nos demandes). J'ai donc été chargé de développer un programme **RPA** en **Python** pour récupérer ces **PDF** nous-mêmes. Le programme tourne tous les soirs pour télécharger une liste prédéfinie de documents. Il les centralise ensuite sur l'espace commun Atlas.

2. Architecture de la solution

J'ai structuré le code pour qu'il fonctionne comme un vrai service autonome et pas seulement comme un petit script.

A. Vérifications de départ et gestion des droits

Avant le lancement des opérations, le script sécurise son environnement d'exécution :

- J'ai utilisé la bibliothèque "**winreg**" pour interagir avec le registre **Windows**. Mon but était de modifier le programme par défaut pour ouvrir les **PDF**, mais comme je n'avais pas les permissions administrateur, je n'ai pu que le lire.
- Le programme vérifie donc si **Adobe Acrobat** est bien le lecteur par défaut. Si ce n'est pas le cas, il s'arrête et affiche un tutoriel pour que l'utilisateur le mette par défaut lui-même. Je ne voulais pas coder la gestion de chaque lecteur **PDF** au cas par cas, donc j'ai fait ce choix pour garder un code simple
- J'ai aussi mis en place un système d'arrêt d'urgence. J'ai utilisé un *listener* asynchrone avec "**pynput**" pour surveiller le clavier. Si l'utilisateur fait "**Ctrl+Shift+Q**", le programme s'arrête instantanément.

B. Mode autonome et configuration

Le programme est conçu pour s'auto-gérer en cas de planification nocturne.

- J'ai utilisé une version de **Chrome** portable pour isoler mon script du navigateur par défaut. Je l'ai aussi configuré en mode "*headless*" (sans interface graphique). Le programme tourne en fond de manière transparente. Comme ça, il n'interfère pas avec l'affichage et reste indépendant de l'**OS**.
- Le programme utilise la fonction "*inputtimeout*" lors du démarrage. S'il possède déjà une configuration valide, il demandera à l'utilisateur s'il souhaite modifier les paramètres actuels et lui laissera un délai de **10 secondes**. S'il n'y a pas de réponse, le programme considère cela comme un non et bascule automatiquement en mode exécution.
- J'ai isolé les données sensibles (**NNI**, mot de passe) et les chemins de répertoires dans un fichier "*config.json*" (créé automatiquement) à la racine du programme. Cela permet à l'utilisateur de modifier ses répertoires ou ses identifiants directement, sans avoir à toucher au code **Python**.

C. Moteur RPA, résilience et fichiers composites

L'interface de Documentum était instable. J'ai donc dû intégrer plusieurs sécurités au pilotage **Selenium**.

- Le programme commence par lire les fichiers **Excel** avec "**openpyxl**" pour extraire les *hyperliens*. Il applique ensuite la méthode **set()** pour ignorer les liens redondants. Comme ça, il supprime les doublons et optimise le traitement.
- J'ai créé une méthode "**resilient_action**" pour encapsuler chaque clic ou saisie. Elle fait des tentatives (*retry*) pour intercepter les exceptions, principalement "**StaleElementReferenceException**"
- Le script gère aussi les ralentissements réseau. Il détecte les barres de progression de téléchargement (**percent**). Il force une mise en attente jusqu'à leur disparition (**staleness_of**) pendant un maximum de **120 secondes**, pour éviter de corrompre le fichier.
- J'ai dû coder une logique spécifique pour les fichiers composites. Le programme identifie ces fichiers dynamiquement, puis il descend dans les "**iframes**" (**view** -> **workarea** -> **content**) et navigue dans les menus contextuels pour faire le téléchargement.
- Enfin, j'ai utilisé la bibliothèque "**ctypes**" de l'**API Win32**. Le script envoie des messages de bas niveau (**WM_CLOSE**, **WM_DESTROY**) pour fermer proprement les pop-up générées par **Adobe** sans gêner l'utilisateur.

D. Maintenance disque et Audit de production

Pour garder un espace propre, le programme gère le cycle de vie des données.

- À chaque lancement, il purge le dossier temporaire de **Documentum**. Il déplace les nouvelles archives et sauvegarde les anciens exports dans un dossier de backup.
- En fin de processus, il injecte un résumé statistique précis dans les logs (console et fichier texte). Il détaille le total traité, les succès, les refus liés aux permissions, et les erreurs techniques.
- Pour vérifier la cohérence, le programme compare mathématiquement le nombre de liens uniques scannés au nombre de fichiers **PDF** physiquement présents. Toute différence déclenche une alerte pour signaler des fichiers manquants ou des doublons.

3. Preuves matérielles

Fonction de résilience :

```
def resilient_action(driver, by, value, action="click", text=None, timeout=30):
    wait = wait_x(driver, timeout)
    locator = (by, value)

    for i in range(50):
        try:
            if action == "text":
                return wait.until(EC.visibility_of_element_located(locator)).text.strip()
            elif action == "click":
                wait.until(EC.element_to_be_clickable(locator)).click()
                return True
            elif action == "send_keys":
                elem = wait.until(EC.element_to_be_clickable(locator))
                elem.clear()
                elem.send_keys(text)
                return True
        except (StaleElementReferenceException,
                TimeoutException,
                ElementNotInteractableException,
                ElementClickInterceptedException) as e:
            if i == 49:
                raise e
            time.sleep(0.5)
    return None
```

Adaptation si le téléchargement prend trop de temps :

```
except (TimeoutException, StaleElementReferenceException, WebDriverException) as e:
    # Gestion des erreurs et retry
    try:
        dlBars = driver.find_elements(By.NAME, "percent")
        if len(dlBars) > 0:
            WebDriverWait(driver, 120).until(EC.staleness_of(dlBars[0]))
            continue
    except TimeoutException:
        pass

    errorName = type(e).__name__
    print(f"Erreur {errorName} : Tentative {attempt + 1}/30")
    cleanErrorPopup(driver)
    attempt += 1
    time.sleep(10)
    if attempt < 30:
        driver.refresh()
        continue
```

Figure 1 : Implémentation du pattern de résilience et d'asynchronisme

Fonction qui gère la fermeture d'**Adobe**, et des pop-up si nécessaire :

```
def killAdobe():
    adobeLilFriends = ["Acrobat.exe", "AcroRd32.exe", "AcroCEF.exe"]
    for process in adobeLilFriends:
        try:
            subprocess.call(
                ["taskkill", "/f", "/t", "/im", process],
                stdout=subprocess.DEVNULL,
                stderr=subprocess.DEVNULL)
        except Exception:
            pass
        try:
            EnumWindows = ctypes.windll.user32.EnumWindows
            GetClassName = ctypes.windll.user32.GetClassNameW
            GetWindowText = ctypes.windll.user32.GetWindowTextW
            PostMessage = ctypes.windll.user32.PostMessageW
            # Signaux de destruction
            WM_CLOSE = 0x0010 # Demande de fermeture
            WM_QUIT = 0x0012 # Ordre de quitter le thread
            WM_DESTROY = 0x0002 # Ordre de destruction mémoire
            def foreach_window(hwnd, lParam):
                cls = ctypes.create_unicode_buffer(256)
                GetClassName(hwnd, cls, 256)
                # #32770 = Boîte de dialogue windows
                if cls.value == "#32770":
                    title = ctypes.create_unicode_buffer(256)
                    GetWindowText(hwnd, title, 256)
                    # Si le titre contient "acro"
                    if "acro" in title.value.lower():
                        PostMessage(hwnd, WM_CLOSE, 0, 0)
                        PostMessage(hwnd, WM_QUIT, 0, 0)
                        PostMessage(hwnd, WM_DESTROY, 0, 0)
                    return True
            # Scan global
            EnumWindows(ctypes.WINFUNCTYPE(ctypes.c_bool, ctypes.c_int, ctypes.c_int)(foreach_window), 0)
        except Exception:
            pass
```

Figure 2 : Pilotage système et fermeture propre

Logs final une fois les tâches terminé (elle est aussi affiché dans la console) :

```
2026-06-26 12:36:31 | INFO |  
=====
```

RÉSUMÉ DES OPÉRATIONS	
=====	
Total des cas traités : 1024	
SUCCÈS : 929	
> Fichiers normaux	: 870
> Fichiers composites	: 59
REFUS (Permissions) : 95	
> Refus globaux	: 95
> Refus composites	: 0
ERREURS TECHNIQUES : 0	
> Erreurs normales	: 0
> Erreurs composites	: 0
> Erreurs fatales	: 0
DURÉE : 1:34:46	
=====	

```
2026-06-26 12:36:34 | INFO | ALERTE : Il manque 95 fichier(s). Consultez les logs.
```

Figure 3 : Rapport d'audit de production

4. Bilan

J'ai pu déployer une solution logicielle robuste. J'ai développé une architecture tolérante aux pannes avec les *retry*, la déduplication et les validations mathématiques. J'ai aussi fait en sorte qu'elle soit respectueuse de l'écosystème **Windows** avec l'**API Win32** et le mode *headless*. J'ai ainsi livré un outil de production autonome qui permet d'assurer la disponibilité des documents pour le projet **Doc-Sat**.